



Module 29

Overview of Formal Analysis Use Models

cā dence[®]

In this module, we're going to introduce formal analysis, what formal does, how it works in general principle and the things you have to take care of in order to run the tool, and how you analyze the results.

Objectives

In this module, you will:

- Differentiate Formal Verification from simulation
- Use Formal Analysis for design exploration
- Use Formal Analysis to prove user-defined functional properties
- Use Formal Analysis to perform Deep Bug Hunting (DBH)
- Understand how selected JasperGold® Formal Verification apps obtain verification closure



This page does not contain notes.

What Is Formal Verification?

Why is it called `formal`?

- This derives from `formal` logic

Reasoning based on manipulation of formulas:

- As opposed to intuition

The name `formal` can mean many things, e.g., any of the following:

- Logic equivalency checking (LEC)
- Model Checking – Functional formal verification (Formal Analysis)
- Theorem proving

This course deals only with Formal Analysis (FA)

3

© Cadence Design Systems, Inc. All rights reserved.



Formal techniques have been around for decades.

It has lately become a mainstream verification method for these reasons:

1. Industry standard languages for defining properties.
2. Improvements in algorithms can yield results in practical timeframes
3. Software tools have become much more user friendly
4. CPUs become much more powerful and less expensive- and memory too
5. Widespread availability of commercial Distributed Resource Management software, e.g., LSF, Sun/Oracle Grid Engine, etc.

Firstly, then formal verification, why is it called formal? This is derived from formal logic, which is based upon manipulation of formulas. So basically, it's just algebra. And this is opposed to simulation where what you're doing is you're guessing. A lot of people don't like admitting this, but when you're doing a simulation, you're just guessing that if you apply certain kinds of stimulus and observe the results, that you've verified your design, but you never know with simulation because you can't simulate every possible scenario. I think most people doing simulation understand that. You can't possibly test everything in simulation because there's too many different cases to test in any reasonable kind of time frame. Now in itself, the name formal can mean many different things to different people. Logic Equivalence checking, which has been around for a long time, this is an example of formal, so this is an algebraic comparison of one design against another. For example, two versions of an RTL design, one of which has been optimized with Clock gating, for example, they're improving, which is there are no commercial tools which do this. This is more of an academic thing. There are tools, but they're all academic ones. This requires a lot of expertise and knowing where you want to end up with. With there and proving basically you propose a theorem and then you propose an addition to that theorem which you try and prove is correct. And if it is correct, then that becomes your new theorem, and you just carry on like that building one thing on top of another until you prove what you really intended to. You have to know where you're going, and it takes a lot of expertise. Now, model checking. This is what all commercial tools do. JasperGold is a model checking tool. We'll talk about what that means as we go through. In order to distinguish model checking from other different kinds of formal, we're going to call model checking formal analysis.

Formal Verification Practical Issues

Writing formal (symbolic) specifications is difficult

How do I know whether I've specified:

- What do I want?
- Everything I want?
- The environment correctly?

Tools and recommended methodologies can help

These are general verification problems

- Not limited to formal analysis



Practical issues are it's difficult to know whether you specified what you really intended, everything you should have, and whether you've set the environment up correctly. However, the tool itself and any recommended methodologies help you through that. And this is what this course is about, really showing you that methodology. And these aren't really problems that are specific to formal that you could apply this to any kind of UVM Testbench as well, for example. These aren't new verification problems. They're problems that any formal verification will have.

SVA Verification Directives

Property

Expression of
design behavior.

- A design behavior defined as logical and temporal relationships between boolean and sequential expressions.

Verification Directive **Keywords**

What the tool should do
with the expressed
design behavior.

- **assert** – An assertion
 - The tool must check that the property is always true.
- **assume** – An assumption/constraint
 - In Formal – The tool must only allow the DUT and inputs to behave in ways that never violate the property.
 - In Simulation – Constraints have no meaning in this context, so it is automatically converted into an assertion.
- **cover** – A cover expression
 - In Formal – Demonstrate that it is possible for the expression to be true.
 - In Simulation – Count how many times the expression is true.
- **restrict** – A readability intent statement
 - In Formal – Exactly the same as **assume**.
 - In Simulation – Has no meaning, hence, is ignored.

The stated design
behaviors in the SVA
properties do not care if
we are a simulation or
formal user.

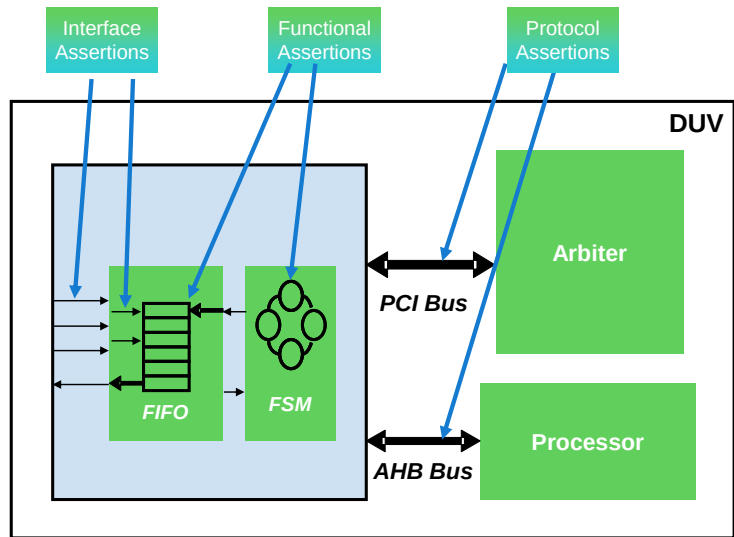
So that's all a property is a definition of behavior, and the verification directives we have in SVA or ASSERT, which means the tool must check the properties are always true. Notice none of these verification directives are specific for formal or simulation. What happens depends upon the tool you use in the course, but there is no definition in the language reference manual of whether you're doing a simulation or whether using the form. The assumption is a constraint, so in formal, what that means is the tool is only allowed to drive the inputs in ways that don't violate that property. Remember, these directives are applied to the property itself in simulation has no meaning because the stimulus is determined by the testbench. What happens in simulators? The Cadence Xcelium simulator is that these will be automatically converted into assertions. A cover in formal is an example of something being true. So normally when you write a cover, you're writing it on a sequence. In formal, when you say to cover a sequence, you're saying to the tool, show me an example of that sequence occurring, if it's possible. In simulation, it means count how many times that expression is true. If the cover was on a sequence, you would count how many times you observe that sequence during the simulation you did. Restrict is not widely used. It's a readability intent statement. In formal, it means literally exactly the same thing as assume. In simulation it has no meaning, so it's ignored. So basically, a restriction is there to use as an assumption in formal for reasons of verification efficiency in terms of throughput, it's not expected design behavior in real life. So really you can view a restriction as a temporary restriction which is only there for convenience of verification, nothing to do with the design behavior required.

Dynamic Verification (Simulation)

Assertions monitor and report:

- Expected behavior
- Forbidden behavior
- Signal protocols

Assertions are dependent on the effectiveness of stimulation vectors.



If we remind ourselves what happens in simulation, then we write these assertions about things like, for example, interface behavior and our protocols being followed correctly. Functional assertions, like all my FIFO pointers working correctly, are my state machine making transitions through the states as I expect. And for this kind of process, which are industry standards, you will probably be using verification IP because someone's already taken the time and trouble to create assertions and assumptions and covers for those protocols. So basically, you just bind that verification IP to that bus, and then you observe the results. If you use a simulation, then how effective your assertions are dependent upon the stimulus. If, for example, you have some assertions written about AHB bus and your stimulus never causes any transactions on the AHB bus, you've never learned anything from that. You've had no failures. But that doesn't mean there's no problem because you never exercised it. In order to know you exercised it, you also need coverage, which we talk about in more detail.

Formal Property Checking

Properties are created from the design specification and implementation decisions:

- *Assertions* are properties that the tool must prove.
- *Assumptions* (a.k.a. Constraints) are properties that inform the tool how the design inputs can behave.

Formal engines verify whether the assertions hold under all input conditions allowed by constraints.

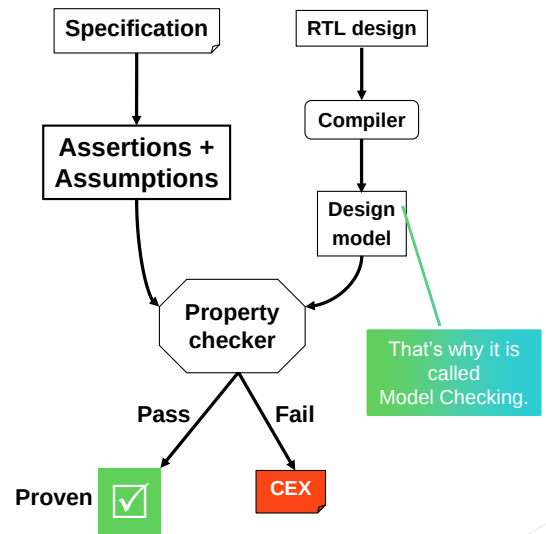
- There is no testbench driving input stimulus.

On a failure, the formal engine should provide vectors to demonstrate failure.

- Known as a Counter Example (CEX).

Properties can be defined with SVA, PSL or OVL.

- Although the vast majority (98%+) of people use SVA.



If we're to draw a kind of flowchart very simplistic, one of which is formal verification, we will do this. From the specification we derive the assertions and assumptions we need. We create our RTL design. This has to pass through the compiler in the JasperGold tool, and from that it creates a model of your design, and then on a one-by-one basis, each assertion is proven against the design model. Is the assertion always true against the design model, using the assumptions we have? And there are two outcomes either. Yes, that assertion is always true, and that's called a proven result, or no, it's not true, and what you get is a counter-example, a waveform showing you a way in which the assertion is violated by the design and the assumptions. There's no testbench there. There's no random stimulus or anything like this. It's an algebraic proof. Is that true under all conditions? And the answer is yes. Whether it is a proven result or not, it's not. Here's a counter-example and we have to debug that counterexample because it's a waveform to understand how does my design get into a state where the assertions violated the properties that are normally defined using SVA? There are other languages that do the same thing. But most people, I'm talking about almost a hundred percent of people use SVA. That has become the dominant language. This creation of the design model here this process is actually known as synthesis in JasperGold. Not like the synthesis you might know as a designer. It doesn't mean you turn it into silicon gates from a library. It means you create a mathematical model. And how would you break the model of the design? And you measure each assertion, whether it's true or not, against that design. So that's why it's called model checking because you have that design model.

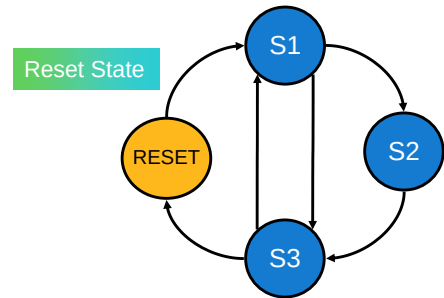
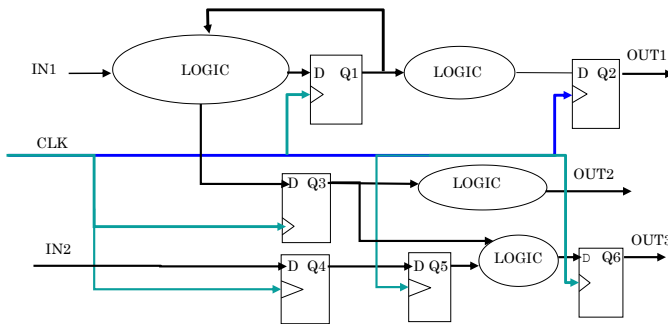
What Is Your Design?

One big state machine

Output of memory elements determines state of the design

State on next clock depends *only* on:

- Inputs
- Current state
- The design



State	Q1	Q2	Q3	Q4	Q5	Q6
Reset	1	0	0	1	1	1
S1	0	1	0	1	0	0
S2	1	0	1	0	0	1
S3	1	0	0	1	1	0

8 © Cadence Design Systems, Inc. All rights reserved.

cadence

- You can view your entire design as being one large state machine. Each memory element, either a latch or a flip-flop, remembers a value.
- If you take an aggregate of all of the memory elements in the design, then this represents the state that the design is in.
- In a design with only flip-flops, the state of the entire design can change on every active clock edge.
- The state that the design goes into next is determined by only three things:
 - The state that you are currently in.
 - The values of the inputs.
 - The design itself.
- The combinational logic is always going to produce the same output, given the same inputs. Outputs change in response to inputs immediately as far as we are concerned for functional verification. Remember, formal analysis is about functionality, not timing.

When this model is built, then how does this viewed by the tool as far as the JasperGold tool is concerned, your design is just one giant state machine and it's the output of the memory elements which determine the state of the design. The memory elements will be the flip-flops, the latches and the memory cells. And for this giant state machine, the state on the next clock depends on what the inputs are doing, what the current state is, and how the design is defined. If you wished and there's no point doing it, but if you wished, you could draw a giant state map of the entire design, which you can imagine will be absolutely massive for a complicated design, and there'll be no point doing it anyway. But you can imagine that that's how your design is viewed by the tool.

How Many States?

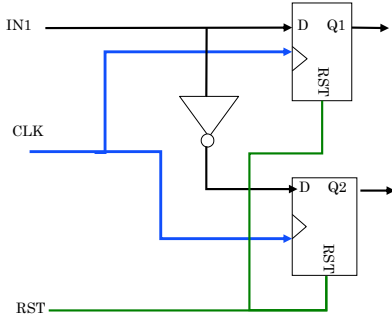
Most probably $\ll 2^{(\text{Number of FFs})}$

Depends on the design

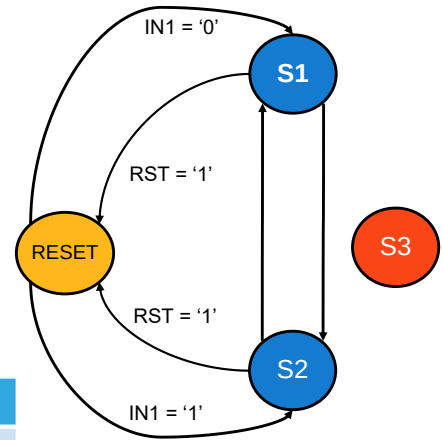
Some states may never be reached

S3 is always unreachable

- If you start from S1, S2 or RESET



State	Q1	Q2
RESET	0	0
S1	0	1
S2	1	0
S3	1	1



9 © Cadence Design Systems, Inc. All rights reserved.



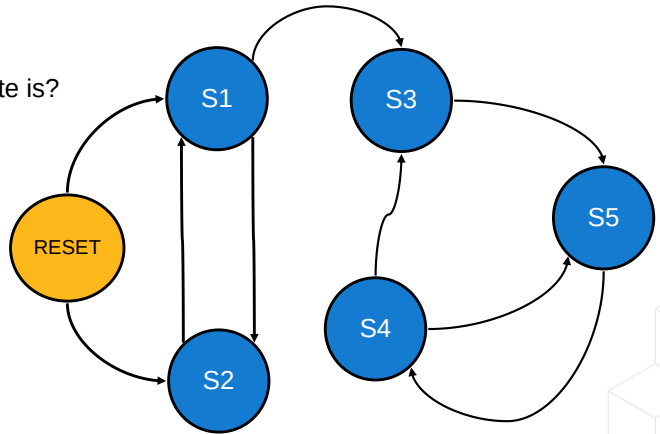
- It is highly unlikely, though possible, that the number of states that your design can be in, is two raised to the power of the number of FFs in your design.
- Your design itself determines how many states it can be in.
- In the example, Q1 can never equal Q2 after the first active clock edge has passed unless RST is active.
- The state where Q1 = '1' and Q2 = '1' can only occur if you initialize the FFs to those values. In verification, that can be done by way of initial signal assignments. In hardware, this can be if your specific technology has FFs that can power up with the Q output set to '1'.
- After an active RST or clock edge occurs, that state is never be reached again.

How many states are there? Well, it's most likely that there's a lot less than two to the power of the number of flip-flops in your design, because it depends on the design. And this example shows here an example of a design where there's only two flip-flops here and there's an inverter between the inputs of these two. If we imagine that we start from some known state, which is a reset state where both outputs of the flip-flops are zero, then there is no way you can ever observe both outputs being one. So not all states are reachable depending upon the design, and there's less states than two to the power of n. This state three here, this is orphaned. We can't ever reach there.

Unreachable States

- Which states are unreachable may depend on the initial state.
- Which state you decide to start from:
 - If initial state is S3, S4, S5, then S1, S2, RESET is unreachable.
 - If initial state is RESET, then all states are reachable.
- How does the verification tool know what the initial state is?

We have to decide what the initial state is.



10 © Cadence Design Systems, Inc. All rights reserved.

cadence

- It is not necessarily true that a state is always unreachable. It can depend upon where you start from.
- In the example, if the design ever enters state S3, then it never goes to state RESET, S1 and S2, again.
- If your initial state for verification is S3, then this means that RESET, S1, and S2 are never reached during that verification run.
- However, if you start from state RESET, then all states are reachable during that verification run.
- The verification engineer has to inform the verification tool of the initial state of the design for a particular verification run.
- `<<states, reachable>>` `<<states, unreachable>>`

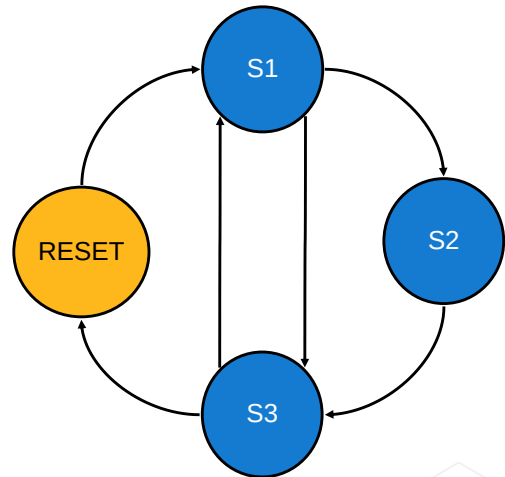
Which states are unreachable and which are not depend upon where you start from, which is called the initial state in formal. If in this example we start from the reset state, all of the states are reachable. There is a path to each other state. If we decided to start verification from state S3, for example, then only states three, four and five would be reachable. There's no path back to these two states, S1 and S2. The question is then how does the tool know where to start from, what the initial state is? And the answer is we'll tell it. We need to tell JasperGold what the initial state is. It's not always the reset state, by the way, either. It could be and often is, but it can be any state you wish.

Changes of State

For every state a finite number of states can be reached on the next cycle.

In this example:

- All states are reachable.
- From state S2, only S3 can be reached on the next cycle.



11 © Cadence Design Systems, Inc. All rights reserved.



- In most cases, only a finite number of states can be reached on the next clock edge.
- The example shows all states are reachable.
- This does not necessarily have to be true. There can be unreachable states.

For every state that we have in this giant state machine, there are a finite number of states we can reach on the next clock. So, in this particular example, all states are reachable. If I'm in state two, I can only reach state three on the next clock.

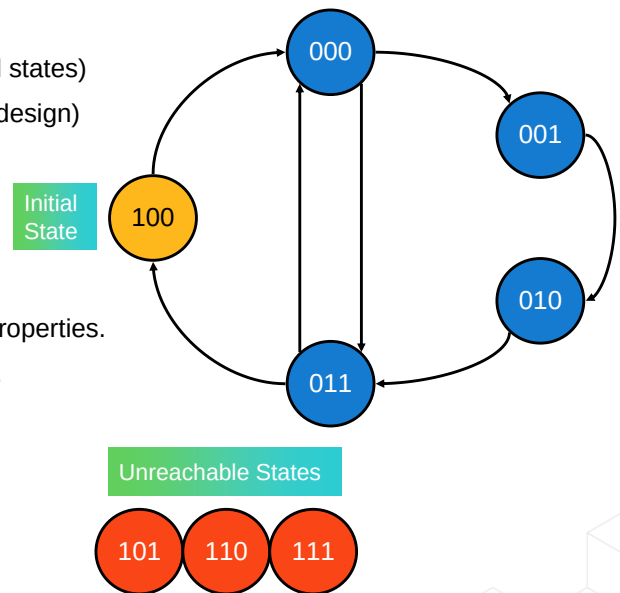
Terminology

Key terms

- State space = 8 (As there are 3 state elements, 2^3 total states)
- Reachable state space = 5 (5 being determined by the design)
- Diameter = 3
 - How many steps from initial state to all reachable states.

Observations

- Reachability and diameter depend on the initial state.
- Verification complexity depends on the design *and* the properties.
- Assertions pass if they are true in every *reachable* state.



12 © Cadence Design Systems, Inc. All rights reserved.



State space. How many possible states that the design can be in? Determined by the number of storage elements in your design.

Effectively, you can view your entire design as being one big state machine.

Reachable state space is how many of the possible states our design can ever be in.

Some states may be unreachable by design; that is, the number of states that your design can ever be in is less than two state space states.

Diameter definition: number of steps it takes to reach all the states in the system starting from the initial set of states.

A good estimate of the diameter of a design is the largest value a counter in the design can count to.

Get to the initial state by the constraints you give the formal analysis tool.

For example, typically, the design is put into the RESET state in a tool-specific way.

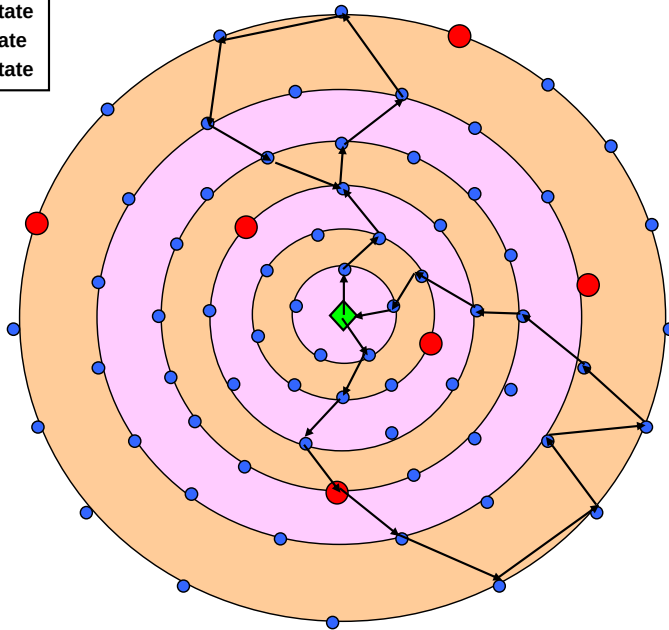
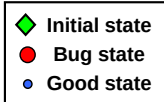
There is another design methodology called *hybrid verification*, where the initial state of the design considered for formal analysis is achieved by running some simulation vectors. This is useful in situations where describing constraints for a static design is extremely difficult.

<<state space>> <<diameter>>

Key terms about this model, then are state space as if there are three state elements. Then that means that I've got two to the three total states which are possible by design for the reasons we saw on previous slides. There's only a much smaller number of states which are actually reachable. In this case we're just arbitrarily choosing the value. Five. And another crucial thing is the diameter. What the diameter means is from the initial state, which we choose, how many steps does it take to reach every other state? In this example, the answer is three, because if the initial state is here, it takes me one, two, three clocks to get to every other state I can reach every other state in three clocks or less.

What we can see from that is reachability and diameter depend upon what the initial state is, which we decide how hard it is for the tool which we're calling verification complexity depends on both the design and the properties, both of which we write. Assertions pass if they are true in every reasonable state, bearing in mind in this model we have three states we can't reach. We're never going to verify anything in those states because they're unreachable.

Formal Analysis Versus Simulation



Formal Analysis

- No testbench required
- Even small diameters equivalent to millions of simulation vectors
- Exhaustive
 - Uncovers all possible bugs
 - Reached max depth/diameter
- Undetermined
 - Inconclusive result
 - Reports verification depth

Simulation

- Follows specific paths
 - Might not hit a bug
- Depends on testbench quality
- Limited controllability
 - Difficult to hit all states
- Applicable later in the design cycle

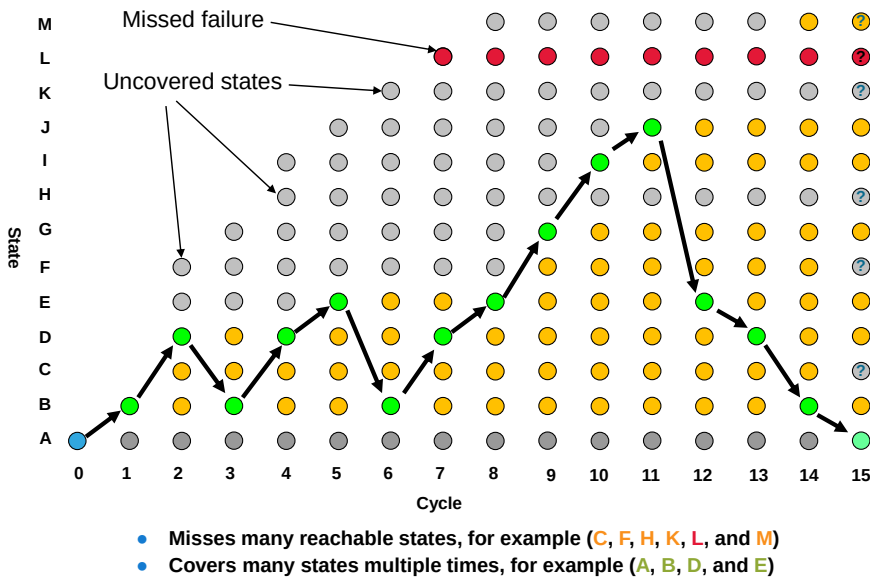
13 © Cadence Design Systems, Inc. All rights reserved.

cadence

Showing a diagram, which nicely shows how formal works and affects. You start from some initial state which we decide, and on the next clock, there will be a finite number of states we can reach on the next clock based upon the design and constraints. And on the next clock, we can reach some more states. There'll be a finite number of states we can reach on the clock after that and on the next cycle there'll be more states we can reach. And this goes on and on and on until we've explored the entire state space.

The benefit of this form analysis is we don't need any testbench. There's no stimulus. Even small diameters. You even making ten clocks might be the equivalent of millions upon millions of simulation test factors. It's exhaustive. If there are any bugs in there, we find them. There's no such thing as corner cases. We will find them. And the problem with this, of course, is that at some stage the state space becomes too big to evaluate in any reasonable kind of time. You might get undetermined results. These red dots represent the bugs; all of them are going to be found by formal contrast with simulation, where what you're doing is starting from some initial state, which is almost certainly the research state. You are making a path through the state space based on what your random stimulus is doing in your testbench. You don't really know what you're doing. Even if you could see this map of states which you can't but if you could, it's still probably not controllable. You couldn't follow that specific path if you wanted to because you wouldn't know what stimulus you needed to do that. Notice you're reaching states more than once so in formal. You only need to visit each state once you know everything about it. Once you've been there in simulation, you tend to keep coming back to the same state and learning nothing new. In these simulations, we run here, two simulations, we followed a path through state space, and we might have hit a bug like we do here. We got lucky here. We hit this bug, but all these other bugs we didn't hit. So that's why people run tens of thousands of simulations runs, of tests and simulations in order to maximize the chances of hitting bugs, but you've got no guarantee. And that's the problem with simulation. You can't test everything, every scenario, and you may or may not hit a bug. And what you hope is that by writing random stimuli and having coverage, you maximize your chances of hitting states that reveal a bug.

Formal Compared with Simulation



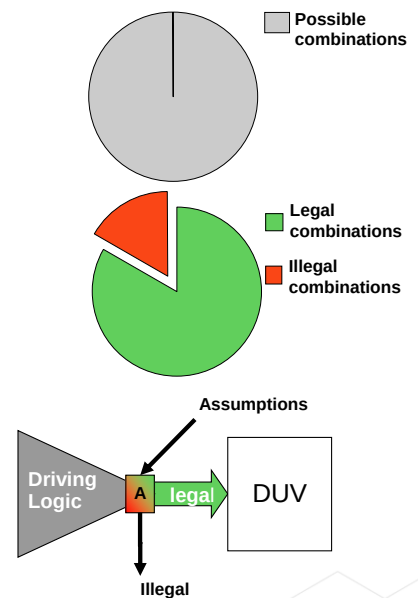
14 © Cadence Design Systems, Inc. All rights reserved.



Another way of viewing that same information is like this. We start from the initial state here, the blue dot, and on the next clock, we see how many new states can we reach. We can reach one more new state here. We don't need to go and visit the state again because we already know everything about that. On the next clock, we can reach four more new states, and on the clock after that, we reach one more new state, and so on, until we've explored all the state space. Within the simulation, what you find is you start from here. You take a path through this state space and you revisiting the same state. It's a state to be visited it one, two, three, or four times. We learned nothing new. After the first time we've been there, we learned nothing new. And notice it's really hard to get to these deep states. It's a really convoluted, specific set of stimuli that will be needed in order to reach these debugs. So that's what's known in formal as bug hunting. When you're reaching these deep parts, that simulation really struggles with. Even if you use lots of different random seeds in tens of thousands of simulations, you still can't get here other than by pure luck.

Assumptions

- Without assumptions, an FA tool checks all possible input combinations of a DUV over time:
 - Likely to get false negatives.
- Arbitrary input combinations are not legal for the proper operation of a design.
- Assumptions model the expectations of the driving logic in a larger context.
- Assumptions specify the legal input combinations of a DUV.
 - Model the verification environment.
- Assumptions exclude illegal input combinations from the verification.
- Assumptions intentionally reduce the State Space.
- Formal tools will **NEVER** violate assumptions.



15 © Cadence Design Systems, Inc. All rights reserved.

Legal inputs are those that we expect to see during normal operation. It is not realistic to expect your design to behave in the correct way when all possible input combinations are being applied unless you explicitly define every possible set of input combinations that your design can theoretically see.

Now for formal, we don't just need the design and our assertions, we also need assumptions. And the reason we need them is because if we didn't have them, we'd get lots of false negatives. These are counter examples which aren't valid ones. If we can imagine all possible input combinations over time with zero assumptions,

what we do is we write assumptions in order to exclude illegal input combinations. In real life you're not expecting your design to work with the inputs driven purely randomly. You know, there has to be some order, you know, inputs have to follow certain protocols in order for the design to behave correctly.

What we're doing with assumptions is modeling the circuit, the environment under which the design is actually working. A formal tool will never violate assumptions. This is crucial to understand this. Those assumptions are taken as axioms by the tool which cannot be questioned. Your tool will never violate the assumptions.

When you think of this as regard our state model, then assumptions are there to intentionally reduce the state space. We don't want to see all these illegal states, which should never happen in real life, because by definition they will never occur.

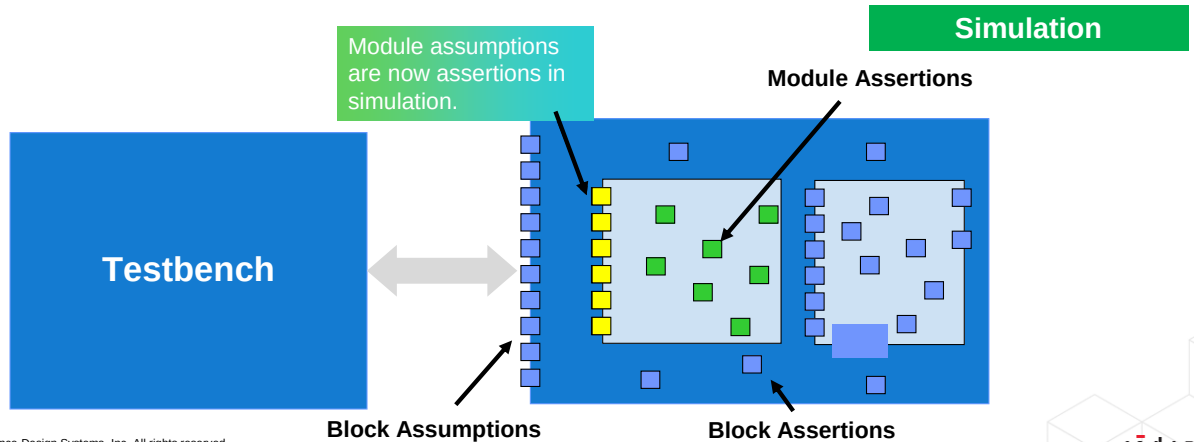
Formal Verification and Simulation of a Block

In Formal Verification, we need both assumptions and assertions, but only assertions are verified.

In simulation, both assertions and assumptions are verified.

The simulator automatically changes assumptions into assertions.

- Allows us to check the validity of assumptions we use in Formal Analysis.



16 © Cadence Design Systems, Inc. All rights reserved.

When we think about this, when we're integrating one block inside of a bigger block and so on until we have the whole system integrated, the assumptions here when we verify this block in formal, we had to write assumptions about what the inputs were doing, how we expect the inputs to behave in real life.

And when we put that block now into a bigger system, what was an assumption here is now convert to an assertion because we need to validate that these properties are still true throughout any simulation test we do. If you can imagine, if we go back here a second, if any of these assumptions are wrong, then all of my formal results could be wrong. If your assumptions are incorrect because they're taken as axioms by the tool, then all your results might be invalid. So that's the reason we need to have another mechanism for verifying assumptions. There are other things we can do as well. Assume guaranteed reasoning, for example, that's beyond the scope of this course.

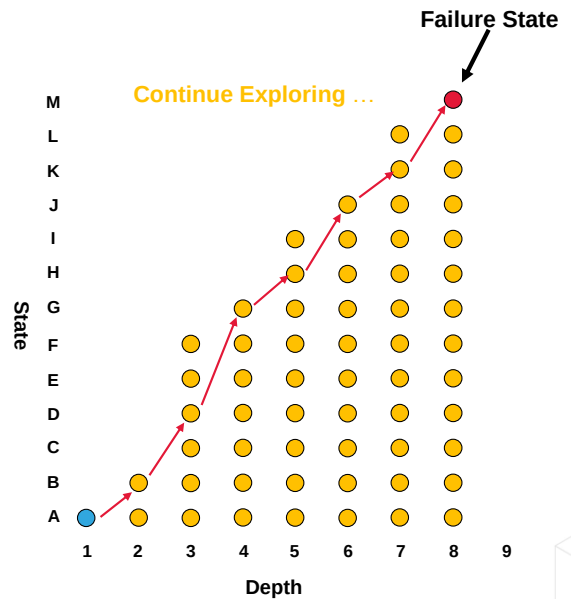
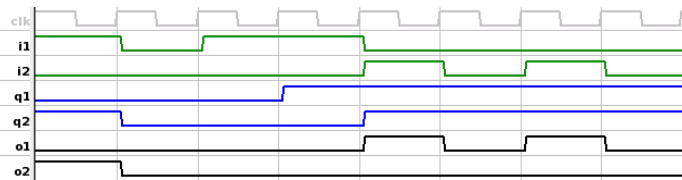
Formal State Space Exploration

Formal visits DUT states as it walks through stimulus

- State = value of all registers/inputs

Formal may reach states that hit properties

- Assertion failures, cover hits
- Converted to waveforms
- Usually, the waveform is the shortest possible path



What does formal do? Going back to the similar diagram I saw earlier; we start from some initial states, and then we continue exploring all different states. How many new states can we reach on the next clocks?

As we go, we might find a state where we observe a failure. What the tool will do is it will draw back the shortest path from the failure back to the initial state and demonstrate that as a waveform to us. So, we have something to debug.

We can also continue exploring and get to the state where we find no failures at all. We know we've visited every state. We know for sure there cannot be a counter-example. So that's an exhaustive proof, a foolproof.

Or continue exploring states, and then the time limit comes in, and we know we've got more states to reach. So that's where the undetermined result comes from. No counterexamples so far, but there may be some in states we haven't yet reached.

Coverage in Formal Analysis

Cover statements express desired reachability of states of the DUV and the environment.

- In formal, with a cover statement, we are asking, “is this cover statement possible to observe?”
- Enables us to detect an unintentionally overconstrained environment.

In simulation, the same cover statement states, “count how many times this cover statement was observed.”

As users of simulation are aware:

- Checks (assertions) without coverage are of little value.

Coverage is as vital in FA as in simulation.



It is important that constraints and properties considered during formal analysis are complete and correct. Check for over-constraining or under-constraining. Check that all inputs and outputs are covered by properties, that is, that there are no holes in the coverage.

It is highly recommended to have more than one person create the properties and constraints. It is also highly recommended that sanity checks be added during the verification.

JasperGold has several tools to automatically check the validity of the user input: Vacuity checks find contradictions in the design or properties and find inputs and outputs that are not covered in formal proofs. This identifies holes in verification coverage. Incisive Formal Verifier looks at cover directives and says if the sequence that is the subject of the cover directive will never happen.

If the sequence can never occur, then the sequence, constraints, or the design must be incorrect.

<<complexity>>

And that's why coverage is as important in formal analysis as it is in simulation. Coverage is essential. Cover statements designed to express the reachability of states, you know, the kind of behaviors we're expecting to be able to see the tool demonstrate to us. So, in formal, a cover statement means Is this cover statement possible to observe and show me a waveform of it happening? This allows us to detect unintentionally over constrained environments. And there are also some tool features which do that as well, which we discuss later. In simulation the same cover, meaning exactly the same code means how many times that cover statement was observed. The same code has different kind of information given to us depend on what kind of tool we're using, although it's the same code. So just like simulation, having checks without coverage is of no value.

Design Exploration

We don't even need to write properties for Cadence's JasperGold Formal Verification to be useful. A cover in formal means "show me an example of this happening."

One does not need to create a testbench or think what stimulus is needed for the cover to be hit.

The target is satisfied WITHOUT the need of a testbench.

We can create this cover with the GUI.

C1: cover property (eg_valid != 1'b0);

The JasperGold Waveform tool is called Visualize and you interact with it, whatever your formal use model is.

19 © Cadence Design Systems, Inc. All rights reserved.

cadence

The first thing is design exploration. This is essential. If you, for example, inherit a new design from a company you've just acquired or you've changed positions in the company, and the first thing you worry about is how can I get up to speed on how this really complicated design works. Well, the answer is

You can use formal. You don't even need to have written any properties for this. If you want to observe particular behaviors of the design, like, for example, producing a burst on the outputs, you don't need to think of what corner case or stimulus do I need to exhibit that behavior. The tool will do it for you, of course, because in formal if you write a cover even manually or using the GUI, what this means is show me an example of this happening. The tool will do whatever it takes in order for this to happen. And if it can happen, then it means your designs are either not capable of it, or your constraints are over-constraining the design inputs in such a way that a burst cannot occur.

Essentially, we can satisfy some given target without the need for a testbench. And if we use the GUI to do this effectively, it's as if we wrote that in our code without having to go through the compilation and editing of files we've already seen Visualize, we talked about that on the previous section and use it in the labs already. This is how you interact with any of the formal apps that we're going to discuss. All the use models, you will be looking at tracers. These are called tracers. These waveforms commonly known as tracers.

Visualize Features: Why?

One can even modify the waveform observed by dragging signal values with a mouse pointer.

Select signal in cycle 5, RMB and select Why?

Relevant Loads

Reason for signal value

Trace Source Browser

The screenshot displays the Cadence Visualize tool interface. The main window shows a waveform with signals like `clk`, `eg_valid`, `eg_ready`, `eg.int_datavalid`, `brdg_rd_ptr`, `eg.eg_ready`, and `brdg_wr_ptr`. A context menu is open over the waveform, showing options like 'Why (find out reason for signal value)', 'Relevant Loads', 'Add Why Signals', 'Highlight Relevant Logic', 'Add Relevant Input/Undriven Signals', 'Add Relevant Module Instance Port Signals', 'Modify Value...', 'Extract Constraint...', 'Add Justify', 'Add Directed Test Point', 'Copy Signal Value', and 'Zoom'. The 'Why' menu item is highlighted. The 'Relevant Loads' menu item is also highlighted. The 'Why' button in the source pane is highlighted. The 'Trace Source Browser' button in the bottom right is highlighted.

You see those, and you interact with them via all the standard things that we've seen so far and described in the previous section, like, for example, the Y feature where you can select any signal in any cycle and see which signal assignment caused that. I won't dwell on that because we talked about this in the previous section. And what it's mentioned in there is wave at it, so you can actually modify what you see to see, can my design exhibit that behavior, or can I still reproduce a counterexample effectively with wave when you're applying constraints to the waveform?

Visualize Features: Highlight Relevant Logic

The screenshot displays the Cadence Visualize interface. On the left, a list of signals is shown, including `a_arbiter_fairness`, `gnt[0]`, `gnt[2]`, `hi_pri[0]`, `hi_pri[2]`, `req[2]`, and `sample_transfer_ready`. A red box highlights a specific signal value in the waveform, and a red arrow points from this value to the 'Highlight Relevant Logic' configuration dialog. The dialog is titled 'Configure Highlight Relevant Logic' and shows a tree view of the design hierarchy with `top (top)` selected. The 'Relevant logic' dropdown is set to 'all', and the 'Color' dropdown is set to 'Auto'. The 'Advanced options' section shows 'Depth' set to 2, 'Show undriven signals only' checked, and 'Show async reset' checked. The 'Do not show this dialog again' checkbox is also checked. The 'OK' button is highlighted.

Highlight Relevant Logic finds all signals responsible for a signal to have a certain value in a cycle.

Configured to search for 2 levels by default.

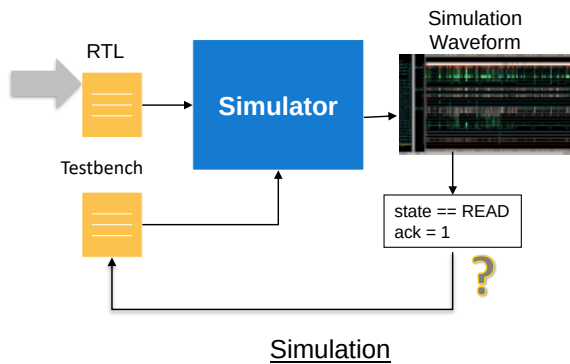
Can be user configured for more levels.

We can also know which signals actually contribute towards a particular signal having a certain value in a certain cycle via analysis of the relevant logic.

This means the logic which drives that signal is in the funding code, basically going back to some number of hierarchical boundaries, which are flip-flop boundaries.

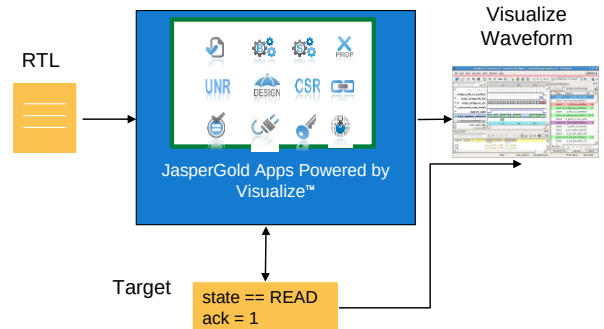
This number of hierarchical boundaries defaults to two. We can of course change that to whatever we all wish because most people using formal have come from doing verification using simulation previously.

Benefits of the Visualize Approach



Pain Points

- “Input driven” trial and error approach to producing the desired behaviors.
- May not exercise desired states.
- Need all elements of the design or must manually create stubs.



Benefits

- Specifies the target and lets the formal engines automatically generate the stimulus (“output driven” method).
- Proof (no trace possible) is exhaustive.
- Interactively adds constraints to construct desired waveform(s).

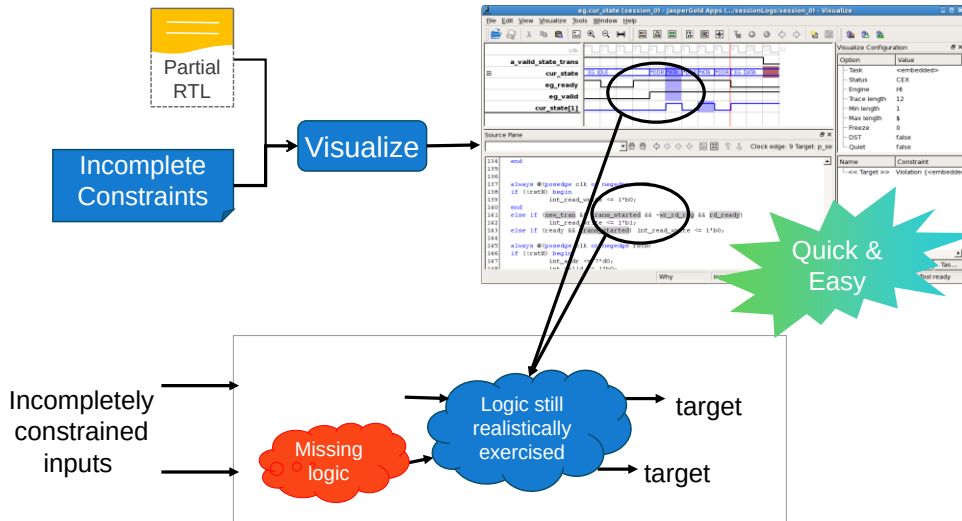
In simulation what we do, whether we like it or not, is we guess what might be a good stimulus. It's all trial and error. We create random stimuli normally in a UVM testbench and apply that to our design and observe what the outputs do in response to that.

The problem with simulation is the first thing you accept if you're doing simulation is you can't test everything and you need the entire design, or you need to go manually and create some stubs or models to act in place there. You can't do it, do it on a partial design. So that's what you're doing in simulation. Basically, these are all the difficulties with it.

With Jasper, however, it's the other way around. It's an output-based, driven method in that what you're doing is debugging counterexamples, which are failures of the properties of your design or inspecting covers in order to explore your design and work out how you ended up in the state where the cover is reached or the counterexample is seen. So basically, we get an end result and then we work backwards to work out how did we get the benefits of this method are that you get the shortest trace possible because that's what formal tools will do, in general. You can interact with the waveforms you're seeing in order to get a different one, to make debugging easier or to experiment with what you think might be RTL fixes and you can know which signals actually affect the result or not.

Partial Designs Can Be Verified

Incomplete RTL or constraints is not a show-stopper for visualize.



23 © Cadence Design Systems, Inc. All rights reserved.



You don't need the whole design because, as we've already seen, each assertion has its own unique COI, and the only part of the design you need in order to verify that property is just what's in the COI. And in many cases, you don't even need the complete COI in order to get a proven result from a property.

You can exercise your design partially, which is important because what this means is you can use formal as a design aid, not just a verification tool. You can use this while you're designing to make sure that what you've just designed literally minutes ago actually does the things, you're expecting it to do without having to think of what corner cases are or without having to think what a testbench is and how that's required and what stimulus you would generate.

What Is Functional Property Verification (FPV)?

FPV is a JasperGold app that most people think of when they hear the term “Formal Verification.”

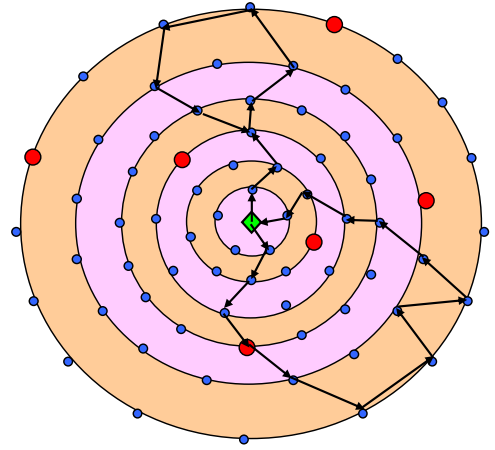
Simply, FPV:

- Writes user-defined SVA functional properties and covers.
- Gets JasperGold to prove exhaustively that all properties are always true and all covers can be hit.
- Debugs any CEXs and fixes them.
- This FPV use model is known as “Classic Formal.”

However, life and formal use models are not that simple.

At some point, the state space becomes too big for formal to prove end-to-end, full-chip, user-defined SVA functional properties.

- Without expert techniques.



24 © Cadence Design Systems, Inc. All rights reserved.

cadence

Now, virtually everything we've talked about so far and what we've done in the labs, and what we will continue to do in the labs is FPV. This is functional property verification. Basically, this means it's user-defined properties, i.e., properties that you wrote and then you wish to prove those inside of Jasper.

So simplistically, this is what we do. We write our functional properties and covers. So, the functional properties mean the asserts and the assumes. We get Jasper to exhaustively prove that all those properties or is true under all circumstances and all covers can be hit. And if we observe counterexamples or unreachable covers, then we go and fix them. This is what's known as classical formal.

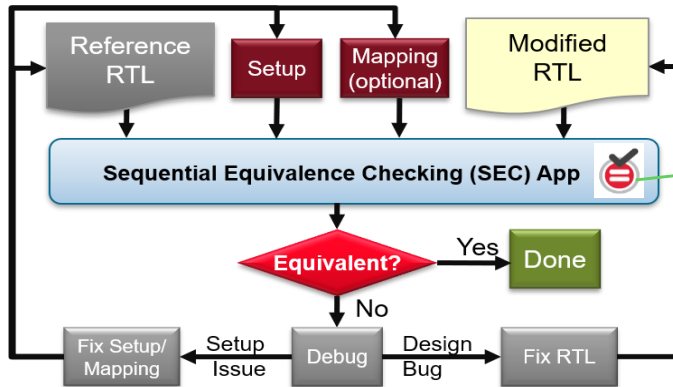
This is almost like a pejorative term, meaning that this is kind of a simplistic way of using it and more advanced users today are using formal for more things like that, particularly what's called deep bug hunting. Getting an exhaustive proof on every single asset you have is not normally a requirement of every formal user. Although it would be nice if you can achieve full proofs on every proper thing.

At some point, your designs will get bigger enough. And remember, the number of transistors you can get in any given area virtually doubles every 18 months. The thing you'll never catch up with is that curve of complexity versus how hard it is to actually verify the state space will always be too big for end-to-end formal checks without some radical discovery in the processing of these formal algorithms. Of course, you can negate that to some extent with expert techniques.

But however, what we're saying here is if you have an entire chip, a big SOC and you write somewhere the assertion that says, if I put these inputs in the design, then this is the output, so I should get out the design. without expert techniques, it's going to be very difficult for the tool to prove that in any realistic time frame. Because the state space is so big, the CLI is likely to be huge. In this use of the classical formal we started at the initial state here and then we jumped out one cycle to all the states that could be reached the next clock and then we jumped to all cycles that can be reached on the clock after that and so on until we've exhaustively covered the state space.

Sequential Equivalence Checking App

Accelerates design convergence



Answers the question:
Are these two versions of the design functionally equivalent?

Sweet-spot use cases:

- Clock Gating Optimization
- Pipeline Retiming

25

© Cadence Design Systems, Inc. All rights reserved.



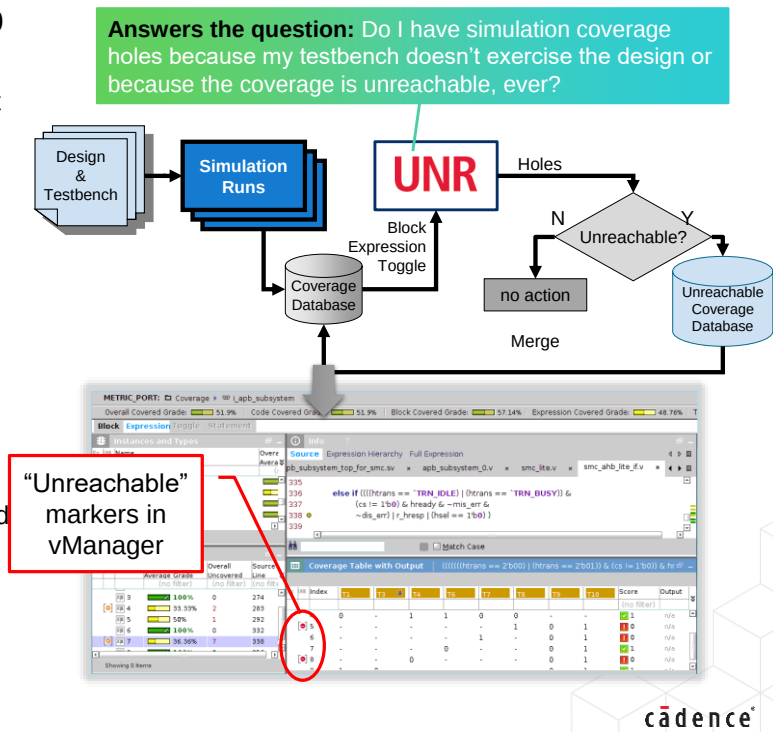
Another very commonly used app is Sequential Equivalence Checking. The difference between Sequential Equivalence Checking and Equivalence checking, i.e., without the sequential on the front, is that this checks the behavior is equivalent at I/Os rather than mapping each internal state element. So that's what sequential means in this term kinds of things people use this for is, when they introduce clock gating to reduce power consumption. The introduction of the clock gating changes the functionality of the design. If they're trying to retime a pipeline, for example, or implement things like multipliers in a different way, is a design still functionally equivalent to what they had before? You compare an implementation with a reference design in effect. You might compare, for example, RTL against gate level and on other kinds of things as well.

So basically, we're trying to check that there are two versions of the design that are functionally equivalent under all circumstances.

Coverage Unreachability App

Saves simulation users weeks of time and effort for verification closure.

- Inputs: Simulation coverage database and RTL.
- Output: Unreachable cover points database.
- Run by simulation users without formal expertise.
- Integrated with vManager to clearly show unreachable coverage points.
- Resilient compilation with Xcelium.
- Supports all Xcelium modeling languages and setup.

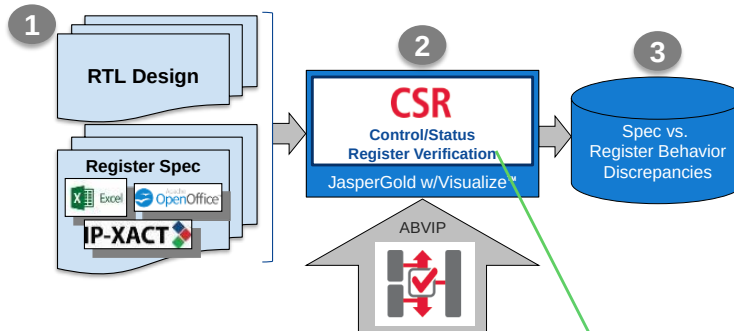


26 © Cadence Design Systems, Inc. All rights reserved.

Another one is another app, and you noticed a feature that a lot of these apps is you don't need to know much about SVA or formal or JasperGold to use them. This app is intended for simulation users. And the reason people use it is because it saves weeks of time typically. As you can imagine, you've got a big suite of simulation tests, and your idea is to get to 100% coverage, and yet you aren't filling your coverage fast enough. Then what you can say to yourself is, is the reason I'm not getting this coverage because the design is never capable of it or that testbench is never capable of driving the stimulus to reach that coverage? Or do I have a bug that prevents the coverage? What happens here, then, is after all your simulation tests, you can dump a database of your coverage, load that into Jasper, and Jasper will create cover statements for the unreachable. And what it will tell you, what Jasper will tell you is, is it possible ever to reach these covers? Because formerly, of course, you can prove that. And if the answer is yes, they are reachable, then you can do two things. You can either sign it off and say yes, it is reachable. I've verified that in formal, or you can modify your testbench, which is not exercised in design in enough ways to reach that coverage. So, either way, you make the decision on what you do next. But the important thing is, you know, whether it's reachable at all or if it's never reachable, this is what Jasper will tell you. So that's the question it's answering. Do I have simulation coverage holes because of my testbench, or do I have some kind of bug that prevents it from ever happening?

CSR Verification App

Exhaustive verification with reduced setup and runtimes.



1. Inputs: Register spec and your RTL design.
2. Run the JasperGold Control/Status Register Verification app.
 - App automatically derives and generates all properties.
 - Automatically runs formal engines to prove all properties.
3. Output: CEXs show discrepancies between the spec and RTL.

Answers the question:

Do all of my register accesses always obey the access policies in my spec?

27 © Cadence Design Systems, Inc. All rights reserved.

cadence

The Control Status Register Verification app is commonly used as well. At the highest level where you're trying to verify your kind of registers, there'll be probably tens of thousands of them.

So normally, you specify these in either some kind of spreadsheet or in IP exact, which is a kind of language for describing these things. And what the tool will do for you is once you've got this exact description; for example, this will guide the access policies. For example, some registers should be readable but not writable, and so on.

What the tool is proving here is generating these transactions in order that you can say, do all of my register accesses, and always obey the access policies defining my spec, which comes from either the IP exact model of that or an Excel spreadsheet.

X-Propagation Verification App

Automatic formal checks with X-Aware analysis and debug.

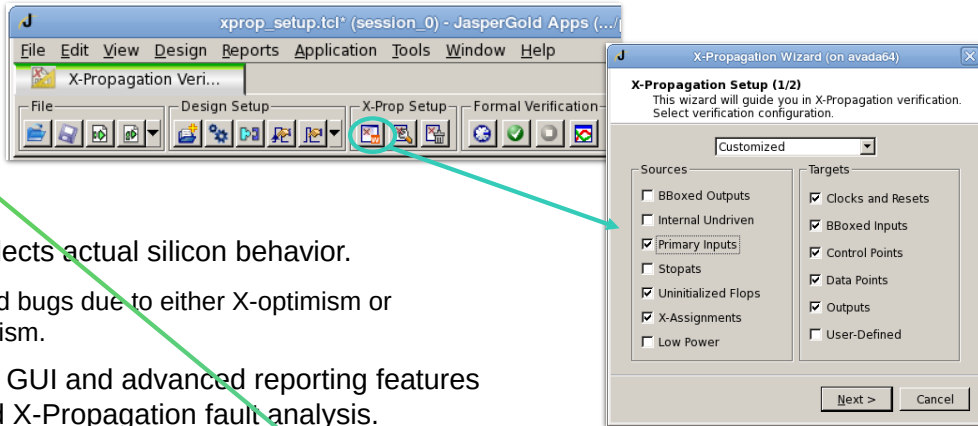


Analysis reflects actual silicon behavior.

- No missed bugs due to either X-optimism or X-Pessimism.

Customized GUI and advanced reporting features enable rapid X-Propagation fault analysis.

Requires no knowledge of formal or SVA.



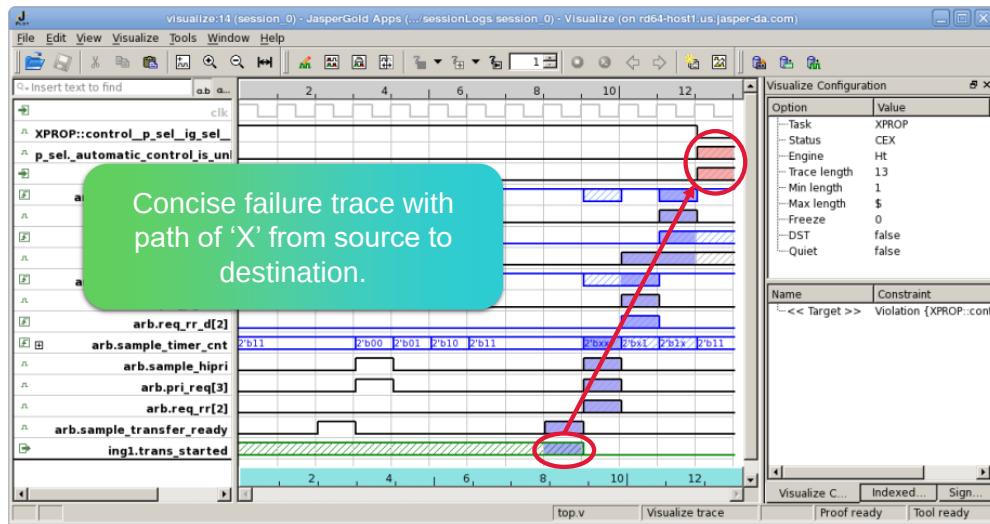
Answers the question:
Can X's reach destinations in my design where I don't want them?

This is an app that we talk about in more detail and also do a lab on.

This is a problem all people can access; reach places in my design I don't wish them to reach.

And the problem with this kind of thing in simulation is you have this concept of pessimism and optimism on X, depending on how X-Propagates, which you need to decide which one do I choose? You don't get this problem in formal because, literally, the value X does not exist. It's either zero or one. It's a set of values that it could be, so formal deals with this as the hardware would. We just select the sources of X that we wish to consider and the destinations we don't want it to reach. And then, the tool will create these checks for us.

XPROP CEX Debug Using Visualize

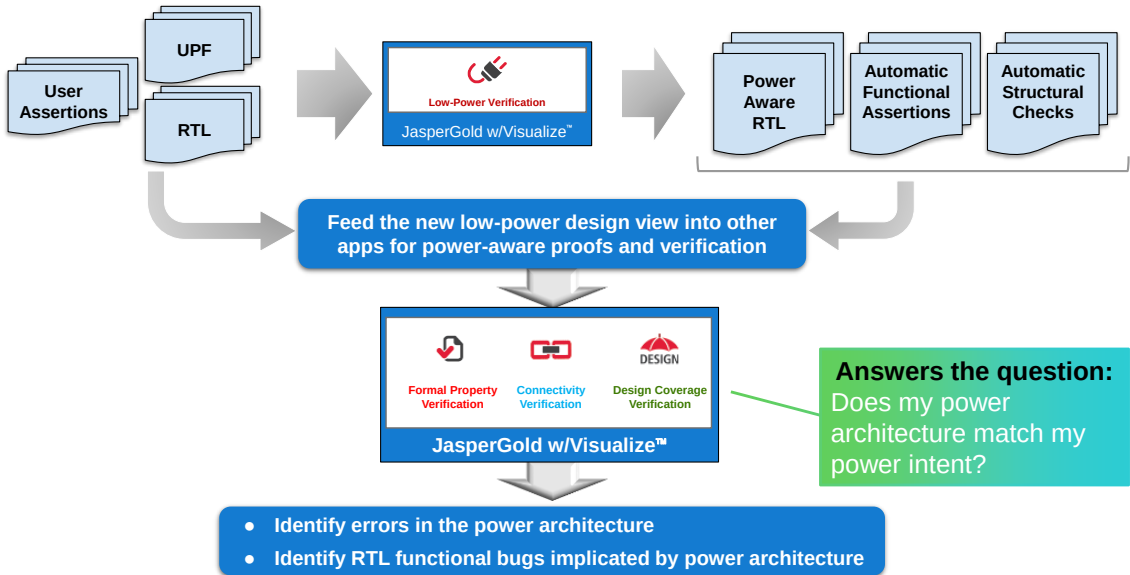


29 © Cadence Design Systems, Inc. All rights reserved.

cadence

And give me a waveform in visualize. And the shading here shows me this is an X value, so I can see an X is reaching this output here, and I can follow back all of the signals that all just and in all the signals where the X is propagating through in order that I can debug this and understand if it's a real problem or not and how to fix it.

Low-Power Verification



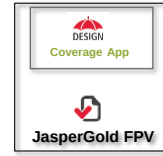
Low-Power Verification, given that you define your power domain architecture using a language called UPF, that's another IEEE standard. Does your actual implementation match that description? This is what the FPV app does. Does my power architecture match my power intent?

Formal Coverage

What Formal-Specific Questions Need to Be Answered?

What code or functionality has been “exercised” by the formal testbench?

- During classic formal or bug hunting
- Covers may be reached, undetermined, unreachable



Answers the question:

Is every part of my design exercised meaningfully by an assertion or a cover?

Is my formal checking complete?

- Properties may be proven, but how much of the design do they cover?
- Complete checks, but not to be fully proven – have the engines achieved sufficient design coverage to detect a bug?

Use JG-COV in conjunction with FPV (or other apps) to answer these questions!



This page does not contain notes.

Summary

In this module, you learned:

- Formal is a very different paradigm from simulation.
- One needs to be prepared for a complete change in mindset.
- Formal does not have a simple “one size fits all” use model.
- New use models are always being created or modified as the technology progresses.
- Apps exist to fix common design and verification problems.
- Many apps are designed with the intent to minimize the amount of SVA and Formal expertise required.
- Each separate app is a specialism in its own right.
 - No one knows all of them in detail.



Obviously, you probably still have a lot of questions from that, but at least you're aware of all the different kinds of things which can be done with formal now.

Remember that formal is a very different paradigm to simulation, and for some of these problems like SPV, equivalence checking, and connectivity checking, you simply can't do them in simulation in any real kind of timeframe.

You need to be prepared for a complete change in mindset.

There's no one size fits all use model. Different customers use different apps, and they're always using different use models anyway.

As time goes on, people find different ways of using these things. Apps are there for the simple reason that they fix common problems people have, and many of these apps are designed with the express intent to minimize the amount of SVA and formal knowledge you need in order to use them, which is why Jasper was so successful.

Each app can get very detailed in its own right, so you can't expect to know how every single app works in great detail.

And in practice, you'll find that you don't need in any given company doing any similar kind of projects. You won't use all the apps necessarily,

There's no right or wrong answer on which apps you should or shouldn't be using; it all depends on what you're doing and how you wish to approach your verification.

This concludes the section on form analysis and models. Now proceed to the next section.